Lamp Installation

Description

Install the following packages and host a sample dynamic website with help of database.

1. Install Apache Webserver
2. Install PHP
3. Install MySQL/MariaDB

Configuration files, extensions and versions should be noticed.

> **Deployment Prerequisites**
> Ensure that your target system has active root privileges (or sudo access), a properly registered subscription or configured repository manager (DNF), and an active network connection to download specific packages.

Step 1 — Install and Configure Apache Web Server

The Apache HTTP Server serves as the foundation for handling incoming client web requests. Execute the following sequence to provision and boot the server application:

1.1 Package Installation

```
yum install httpd -y
```

1.2 Service Management

Initialize and configure the Apache system daemon to execute persistence parameters across system restarts:

```
systemctl start httpd
ssystemctl enable httpd
```

1.3 Verify Initialization Status

```
systemctl status httpd
```

1.4 Infrastructure Directory & Layout Structure

- **Default Document Root:** /var/www/html (The global dynamic context area)
- **Primary Configuration File:** /etc/httpd/conf/httpd.conf
- **Supplementary Module Configurations:** /etc/httpd/conf.d/
- **System Error and Output Logs:** /var/log/httpd/

1.5 Verify Deployment Version

```
httpd -v
```

To confirm active connectivity, navigate your browser to http://SERVER-IP to observe the default Apache welcome interface.

Step 2 — Install MariaDB Database Server

MariaDB provides the relational data storage tier for structural web application metrics.

2.1 Core Package Provisioning

```
yum install mariadb-server -y
```

2.2 Service Activation

```
systemctl start mariadb
ssystemctl enable mariadb
```

2.3 Status and Verification

```
systemctl status mariadb
mysql --version
```

Step 3 — Hardening & Securing MariaDB

Production environments necessitate runtime security posture compliance. Execute the underlying script to eliminate default test assets and restrict unauthorized authentication permissions:

```
mysql_secure_installation
```

During execution, configure parameters interactively following these enterprise hardening best practices:

- **Set root password? Y** (Define an internal complex alpha-numeric string)
- **Remove anonymous users? Y** (Disallows explicit unauthorized client mapping)
- **Disallow root login remotely? Y** (Restricts database administrator access parameters strictly to local context loops)
- **Remove test database? Y** (Purges insecure default schema visibility)
- **Reload privilege tables? Y** (Forces structural memory execution immediately)

Step 4 — Database Initialization & Account Management

Establish the persistence layout for your custom dynamic web platform:

4.1 Root Shell Login

```
mysql -u root -p
```

4.2 Relational Structure Creation

```
CREATE DATABASE sampledb;

CREATE USER 'rejin'@'localhost' IDENTIFIED BY 'rejin123';

GRANT ALL PRIVILEGES ON sampledb.* TO 'rejin'@'localhost';

FLUSH PRIVILEGES;

EXIT;
```

Step 5 — Install PHP and Associated Dependencies

PHP interprets server-side business logic and securely queries data pipelines to present structural interfaces.

5.1 Package Setup with Dependencies

```
dnf install php php-mysqlnd php-cli php-common php-gd php-mbstring php-xml php-json -y
```

5.2 Post-Install Module Validation

- **Core Engine Config:** /etc/php.ini
- **Version Output Check:** php -v

5.3 Engine Synchronization

Restart the HTTP Server infrastructure to trigger deep process links with the newly provisioned PHP compilation layers:

```
systemctl restart httpd
```

Step 6 — Create and Validate Sample Dynamic Website

Establish a real-time responsive functional check to verify internal communication pipelines between your Web Server layer and Database tier. This script includes explicit runtime error debugging configurations alongside an integrated PHP info report payload:

6.1 Target Document Construction

```
nano /var/www/html/index.php
```

6.2 Inject Unified Operational Codeblock

```
<?php

error_reporting(E_ALL);
ini_set('display_errors', 1);

$conn = new mysqli("localhost", "rejin", "rejin123", "sampledb");

```

```
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

echo "<h2>Database Connected Successfully</h2>";

echo "<hr>";

phpinfo();

?>
```

Save and close the document editor. Launch a browser window and process http://SERVER-IP/index.php. If the operational variables, user credentials, and database layouts match correctly, your system will securely report Database Connected Successfully followed by the foundational environment information page.

Technical Reference Appendices

Configuration File Mapping Table

Component / Environment Target	Global File Path Context Location
Apache Base Engine Configuration	/etc/httpd/conf/httpd.conf
Apache Modular Config Directory	/etc/httpd/conf.d/
PHP Operational Context Specification	/etc/php.ini
MariaDB/MySQL Native System Engine Options	/etc/my.cnf

Installed Modules & Extension Mapping

Module Name	Architectural Purpose Context
php-mysqlnd	Native Dynamic MySQL Driver communication management link.
php-cli	Command-line interface processing pipeline executor.
php-gd	Server-side image generation and graphic formatting wrapper.
php-mbstring	Multi-byte character management tool for non-ASCII variables.
php-xml	Extensible Markup Language extraction and query translation.

Firewall Configuration and Utility Commands

To safely pass HTTP requests through external firewalls, expose port boundaries explicitly:

```
firewall-cmd --permanent --add-service=http
firewall-cmd --reload
```

Output Pictures